

Code-breaking Neural Networks

Tristan Hodgson

December 29, 2025

1 Motivation

During this project, I investigate the ability of neural networks to learn to decrypt cipher text. While this is not possible for general encryption schemes (e.g., AES and RSA) which rely on computationally difficult problems, if we restrict ourselves to less cryptographically secure ciphers, it may be possible for neural networks to learn the underlying transformation from examples alone. This project aims to replicate and extend previous work in applying deep learning to cryptanalysis, and explore how known prefixes accelerate the learning process.

2 Data

All data is gathered from the Project Gutenberg[1] corpus of public domain books. Text is preprocessed by converting to lowercase and removing all non-alphabetic characters to standardise the input space to 26 characters (plus special tokens).

Aside from the proof of concept described above, we train using pairs of (key+ciphertext, key+plaintext) generated by applying a cipher to the plaintext during training. The ciphers we consider in this project are the Caesar cipher, the simple Substitution cipher, and finally the Enigma machine.

3 Model Architectures

We experiment with two seq2seq architectures that have been used in the literature: a 1D Convolutional Neural Network (CNN) and a recurrent network based on Long Short-Term Memory (LSTM) units.

3.1 Convolutional Neural Network

The 1D CNN architecture is motivated by its success in ciphertext classification and pattern detection tasks [2]. Convolutional filters can act as detectors for short-range structure, similar to n -gram frequency analysis traditionally used in classical cryptanalysis. For simple ciphers such as Caesar and substitution ciphers, this inductive bias is often sufficient to uncover key-dependent patterns [3]. Our CNN uses filters of sizes 2, 3, 5, and 7 chosen to capture common n -gram lengths in English text.

3.2 Recurrent Neural Network (LSTM)

The second architecture is an LSTM-based recurrent neural network, replicating the work demonstrated by Greydanus et al. [4]. Unlike CNN models, the LSTM is stateful, allowing it to capture long-range dependencies across the ciphertext. This is of particular importance for polyalphabetic ciphers like Enigma, where the internal state of the machine changes with every character processed.

Training is supervised: the model receives ciphertext sequences and must predict the corresponding plaintext tokens. We predict the next plaintext character at each timestep until the end-of-string token is generated.

4 Experimental Variation: Known Prefix

A core challenge in neural cryptanalysis is the model's ability to infer the cipher's internal state (or "key") solely from statistical patterns in the ciphertext. In their seminal work, Greydanus et al. [4] demonstrated that LSTMs can learn algorithmic representations of ciphers, but often require the key to be explicitly provided or inferred from very long sequences.

To investigate the model’s capacity for *known-plaintext attacks*, we introduce a specific variation to the training data: appending a fixed **pangram prefix** to every plaintext sequence prior to encryption.

- **Prefix:** “the quick brown fox jumps over the lazy dog”
- **Mechanism:** This prefix contains every letter of the alphabet at least once. When encrypted with a substitution cipher, the first 43 characters of the ciphertext effectively expose the entire substitution table (the key) to the network.

We hypothesize that the LSTM utilises this prefix to rapidly “load” the correct decryption permutation into its cell memory state. By processing the encrypted pangram, the model can identify the mapping $C \rightarrow P$ for every character in the alphabet. This effectively converts the task from blind statistical cryptanalysis to a dynamic lookup task, where the model learns to persist the state derived from the prefix to decrypt the subsequent message. This mirrors the concept of a “crib” in classical cryptanalysis—a known piece of plaintext used to break the cipher.

Comparing the convergence speed and final accuracy of this “Prefix Model” against the baseline (blind) model allows us to quantify the LSTM’s ability to perform state-dependent algorithmic learning versus pure pattern matching.

5 Future Work

Several extensions could build on this proof of concept:

- **Language Generalisation:** Training the model on different languages (e.g., German) to determine if the learned decryption algorithms are language-dependent or if the model learns the pure logic of the cipher independent of linguistic statistics
- **Mixed Ciphers:** Increasing complexity by randomly switching between two different ciphers (e.g., Caesar and Substitution) for each character in the plaintext, forcing the model to dynamically infer the encryption method at each timestep
- **Key Reconstruction:** Instead of decoding to plaintext, modifying the architecture to output the encryption key itself, testing the model’s ability to explicitly perform key recovery attacks

These extensions would help determine how far neural approaches can be pushed into genuine cipher analysis and more sophisticated symbolic tasks.

References

- [1] Project gutenber. <https://www.gutenberg.org/>. Accessed 2025-11-22.
- [2] Ezat Ahmadzadeh, Hyunil Kim, Ongee Jeong, Namki Kim, and Inkyu Moon. A deep bidirectional lstm-gru network model for automated ciphertext classification. *IEEE Access*, 10:3228–3240, 2022.
- [3] Riccardo Focardi and Flaminia L. Luccio. Neural cryptanalysis of classical ciphers. In *Proceedings of the 3rd Italian Conference on Cybersecurity*, 2018.
- [4] Sam Greydanus. Learning the enigma with recurrent neural networks. *arXiv preprint arXiv:1708.07576*, 2017.